Introduction to GitHub Copilot

Define, inspect, test, decide

Session M1-01 | 3 hours

Agenda

Segment	Time
Suggestion-and-tests model	7 min
How context shapes a suggestion	10 min
Inline suggestion controls	18 min
Prepared utility demonstration	17 min
Follow-up edits	5 min
Lab handoff	3 min

The review workflow

```
Define observable behavior
          ↓
Inspect the suggestion
          ↓
Run focused checks
          ↓
Accept, revise, or reject
```

The developer owns the result.

Plausible is not correct

Looks right	Is right
Reads well	A focused test passes
Uses familiar syntax	The diff stays in scope
Matches one example	Edge and failure cases are checked

A practical system model

```
Task + context
          ↓
Copilot surface
          ↓
Suggestion
          ↓
Tests + human review
```

Context changes the suggestion

Useful context includes:

- function names and type hints;
- docstrings and examples;
- selected code and nearby files;
- tests and repository instructions.

How inline suggestions work

Trigger: type, or pause after a comment or function signature. Copilot shows the suggestion as grey ghost text. You can also request one by hand.

Inline vs Chat: inline completes the code at your cursor as you type. Chat answers a question you ask, and can explain or edit code. Use inline for the next few lines. Use Chat when you need to ask or explain.

Take, change, or dismiss a suggestion

Action	Default in VS Code
Accept all	`Tab`
Accept next word	`Ctrl+Right`
Accept next line	No default key. Use the Command Palette, or bind a key
Cycle alternatives	`Alt+]` and `Alt+[`, or the hover toolbar
Dismiss	`Esc`, or keep typing

Check your version. Shortcuts differ by editor, OS, and version. The editor shows the ones that apply to you.

Suggested code is not tested code

Copilot predicts likely code. It does not run it.

- A suggestion can compile and still be wrong.
- A suggestion can pass the one example you thought of.
- **Only your tests and your review show it is correct.**

Accept, revise, reject

Accept when the proposal meets the contract and fits the repository.

Revise when the direction is useful but a detail is wrong.

Reject when it changes scope, invents behavior, or is hard to check.

Undo is a normal review action.

Review one suggestion

Before accepting, ask:

- Which requirement does each line support?
- What happens on empty or unusual input?
- Did it add a dependency?
- Did it change a caller or public name?
- Which check will fail if the logic is wrong?

Demo contract

```
def normalize_username(value: str) -> str:
    """Trim outer whitespace, lowercase the value,
    and join words with hyphens."""
```

```
assert normalize_username("  River Team  ") == "river-team"
```

The test checks how repeated whitespace is handled.

Demo: inspect before accepting

Review the proposed implementation against three rules:

1. Outer whitespace disappears.
2. Case becomes lowercase.
3. Each run of internal whitespace becomes one hyphen.

Then run the focused test.

A failing test is useful

This version looks reasonable but mishandles repeated spaces:

```
return value.strip().lower().replace(" ", "-")
```

The test catches it while the change is still small.

Predicted follow-up edits still need review

```
Rename username to handle
          ↓
Inspect each proposed caller update
          ↓
Search for the old name
          ↓
Run the focused test
```

A predicted edit can miss a reference.

Lab handoff

You will extend one Python utility module:

1. complete `normalize_username` and `is_palindrome`;
2. decide how apostrophes count in `word_frequency`;
3. add `profile_summary`;
4. rename `username` to `handle`;
5. run the full test suite and review the diff.

Deliverable: completed code, five passing tests, and a decision record.